

```
=== FILE: map_editor/ui/info_panel.py ===
```

```
"""Панель информации о клетке – полностью автоматическая"""
```

```
import pygame
from ..config import COLORS
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG, WEAPON_CONFIG
```

```
class InfoPanel:
    """Отображает информацию о текущей клетке"""

    def __init__(self, rect):
        self.rect = rect
        self.cell_x = None
        self.cell_y = None
        self.symbol = None
        self.tool = 'BRUSH'
        self.selected_symbol = 'M'
        self.has_changes = False
        self.font = pygame.font.Font(None, 16)

    def update(self, cell_pos, symbol):
        if cell_pos:
            self.cell_x, self.cell_y = cell_pos
            self.symbol = symbol
        else:
            self.cell_x = None
            self.cell_y = None
            self.symbol = None

    def update_status(self, tool, selected_symbol, has_changes):
        self.tool = tool
        self.selected_symbol = selected_symbol
        self.has_changes = has_changes

    def draw(self, screen):
        pygame.draw.rect(screen, COLORS['info_bg'], self.rect)

        left_text = ""
        if self.cell_x is not None:
            type_name = self._get_type_name(self.symbol)
            left_text = f"({self.cell_x}, {self.cell_y}) Символ: '{self.symbol}' Тип: {type_name}"
        else:
            left_text = "Наведите на клетку"

        text_left = self.font.render(left_text, True, COLORS['text'])
        screen.blit(text_left, (self.rect.x + 12, self.rect.y + 10))

        right_text = f"Инструмент: {self.tool.upper()} | Объект: '{self.selected_symbol}'"
        if self.has_changes:
            right_text += " | * (изменено)"

        text_right = self.font.render(right_text, True, COLORS['text_dim'])
        right_x = self.rect.right - text_right.get_width() - 12
        screen.blit(text_right, (right_x, self.rect.y + 10))
```

```

def _get_type_name(self, symbol):
    """Полностью автоматическое определение типа объекта"""
    if not symbol:
        return "-"

    # =====
    # 1. ПРОВЕРЯЕМ SYMBOLS_CONFIG
    # =====
    config = SYMBOLS_CONFIG.get(symbol, {})
    symbol_type = config.get('type', '')

    if symbol_type == 'wall':
        return "Стена"

    elif symbol_type == 'door':
        door_type = config.get('door_type', 'normal')
        required_key = config.get('required_key')

        if door_type == 'secret':
            return "Секретная дверь"
        elif required_key:
            return f"Дверь (ключ: {required_key})"
        else:
            return "Дверь"

    elif symbol_type == 'exit':
        return "Выход"

    elif symbol_type == 'player_spawn':
        return "Старт"

    elif symbol_type == 'item':
        item_type = config.get('item_type', '')
        weapon_name = config.get('weapon_name')
        key_color = config.get('key_color')

        if item_type == 'health':
            return f"Аптечка ({config.get('amount', 25)})"
        elif item_type == 'armor':
            return f"Броня ({config.get('amount', 25)})"
        elif item_type == 'key' and key_color:
            return f"Ключ ({key_color})"
        elif weapon_name:
            return f"Оружие: {weapon_name} ({config.get('ammo', 0)})"
        else:
            return "Предмет"

    # =====
    # 2. ПРОВЕРЯЕМ NPC_CONFIG
    # =====
    npc_config = NPC_CONFIG.get(symbol, {})
    if npc_config:
        name = npc_config.get('name', 'NPC')
        class_name = npc_config.get('class_name', '')
        if class_name == 'Boss':
            return f"Босс: {name}"
        else:
            return f"NPC: {name}"

```

```
# =====
# 3. ПРОВЕРЯЕМ WEAPON_CONFIG (если оружие не в SYMBOLS_CONFIG)
# =====
weapon_config = WEAPON_CONFIG.get(symbol, {})
if weapon_config:
    return f"Оружие: {symbol}"

# =====
# 4. НЕИЗВЕСТНЫЙ ТИП
# =====
return "Объект"
```